

FLUTTER

What is Flutter ?

Flutter is a free, open-source framework from Google that helps developers build applications for multiple platforms with a single codebase. It's used to create the user interface (UI) for mobile, web, desktop, and embedded devices. Compared to Android(Kotlin) or iOS(Swift), Flutter is much easier to learn and use, it is also suitable for those who have little to no experience in programming.

Here are some things to know about Flutter:

- Supported platforms: Flutter supports iOS, Android, the web, Windows, MacOS, and Linux
- Programming language: Flutter uses Dart, a typed object programming language with a syntax similar to JavaScript.
- Tools: Flutter includes automated testing, developer tooling, and other tools to help build production-quality apps.
- Pre-built widgets: Flutter comes with a variety of reusable UI elements, such as buttons, sliders, and text inputs.
- Apps Build by Flutter: Google Pay, Alibaba, BMW, Google Earth etc.

Features of Flutter

1. **Hot Reload:** Instantly see the results of changes made to your code without restarting the app, making development faster and more efficient.
2. **Cross-Platform Development:** Write once and deploy on multiple platforms, including iOS, Android, web, and desktop.
3. **High Performance:** Compiles to native ARM code for both iOS and Android, ensuring smooth performance.
4. **Animations and Motion:** Built-in support for rich animations and transitions, allowing for engaging user interfaces.
5. **Testing and Debugging:** Comprehensive testing framework that supports unit, widget, and integration testing.
6. **Responsive Design:** Flexible layouts that adapt to different screen sizes and orientations.

Disadvantages of flutter

1. **Large App Size:** Flutter apps tend to have larger file sizes compared to native apps due to the framework and engine being bundled with the application.
2. **Limited IDE Support:** Although IDEs like Android Studio and Visual Studio Code offer support, it may not be as comprehensive as for some other languages and frameworks.
3. **Longer Build Times:** Depending on the complexity of the app, build times can be longer compared to native development, particularly in large projects.
4. **Limited Web Support:** Flutter's web capabilities are improving, but they may not yet match the functionality and performance of dedicated web frameworks.

How to Install Flutter ?

Step 1: Open your Browser and Navigate to "flutter.dev" on your webpage. On the top menu bar, select Docs > Get Started > Install > Windows.

Step 2: You have to install the latest versions of Windows PowerShell 5.0 and Git for Windows, if not already installed.

Step 3: After installing Git on your windows. Once done, let's get to the installation of Flutter Software development Kit (Flutter SDK). Click on the download link for the latest version (as of today).

Step 4: After the download is complete, extract the 'flutter' folder (drag and drop) to any path/directory of the system where you get the read and write access.(Eg. C: > development > flutter).

Step 5: Now double-click on the 'flutter' folder. Go to 'flutter_console.bat' file and double-click to open a command prompt window, then type "flutter" and press enter.

Step 6: Check and edit environment variables for global system access. for that, copy the path from "flutter/bin" and paste it on environmental variables > press on path > new > save.

Step 7: Open any terminal and type in 'flutter' and see whether you get the same list of commands as you did get earlier from the flutter.bat terminal. If it does not work, you might re-run the setup again.

Step 8: Type "flutter doctor" to check for other requirements for build projects. You can see that flutter was installed successfully in our system, but the Android tools are missing. For that Download and Install Android Studio in your system.

Step 9: After installation, click Next > Choose Custom > Next. Here you can see some checkboxes, you can ignore the shows already installed. Then click on Next > Finish.

Step 10: Accept required Android Licenses. On the Command Prompt terminal, type "flutter doctor --android-licenses" and enter You'll see a couple of repeated prompts that look like this Accept? (y/N), Type 'y' whenever asked for.

Step 11: Finally shows after all the license agreements have been accepted, you should see a message that looks something like "All SDK package licenses accepted".

Now have a fully functional Flutter Framework with devices/emulators to build your beautiful apps on your System.

How to Create a Flutter Project ?

We can create our flutter project in some ways,

Using Visual Studio Code,

1. Open VS Code, then Go to View > Command Palette or press Ctrl / Cmd+Shift+P.
2. Type flutter then select "Flutter: New Project" and Press enter.
3. Select Application then Select a Project location and Enter a Project name.

Using Android Studio,

1. Open Android Studio, click on New Flutter Project from the welcome screen or click on New and then New Flutter Project from the File menu. Android Studio will open the New Project dialog.
2. Then enter the path to your Flutter SDK directory. Verify the path is correct and then hit the Next button.
3. Enter your project name. In the New Project box, enter the project name. No other changes are required. Click on Create.

Using Terminal,

1. First, you have to open your folder that you want to install your Flutter Project
2. Inside the project you have to press right click and open terminal then type "flutter create your_app_name" then enter.
3. Select Application then Select a Project location and Enter a Project name.

To learn Flutter, you need to get to know the Dart programming language, as it's the core language used in the Flutter framework.

Widgets

In Flutter, 'everything is a widget.' Widgets are the basic building blocks of a Flutter app's UI, representing immutable parts of the interface. They describe elements like text and buttons, as well as layout effects like padding and alignment. Widgets are either stateless, which don't hold state, or stateful, which can change based on user interactions. Combining widgets allows for flexible and efficient UI design.

Flutter classifies widgets into two types:

- Stateless Widgets and
- Stateful Widgets.

Stateless Widgets

Stateless Widgets are immutable and do not change over time. They are built once and remain constant during their lifecycle.

Syntax:

```
import 'package:flutter/material.dart';

class MyStatelessWidget extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Text('Hello, Stateless Widget!');
  }
}
```

Stateful Widgets

Stateful Widgets can change state during the app's lifecycle. When their state changes, the widget rebuilds.

Syntax:

```
import 'package:flutter/material.dart';
```

```
class MyStatefulWidget extends StatefulWidget {  
  @override  
  _MyStatefulWidgetState createState() => _MyStatefulWidgetState();  
}
```

```
class _MyStatefulWidgetState extends State<MyStatefulWidget> {  
  int _counter = 0;  
  
  void _incrementCounter() {  
    setState(() {  
      _counter++;  
    });  
  }  
}
```

```
@override  
Widget build(BuildContext context) {  
  return Column(  
    children: <Widget>[  
      Text('Button pressed: $_counter times'),  
      ElevatedButton(  
        onPressed: _incrementCounter,  
        child: Text('Increment'),  
      ),  
    ],  
  );  
}
```

The widget lifecycle is a critical concept in widget development that outlines the various states a widget goes through during its existence. This lifecycle helps developers manage how a widget behaves in response to changes in data, user interactions, and the environment.

Now We have to learn about a topic, which is Widget Lifecycle,

The widget lifecycle is a critical concept in widget development that outlines the various states a widget goes through during its existence. This lifecycle helps developers manage how a widget behaves in response to changes in data, user interactions, and the environment.

Widget LifeCycle

- **Keep It Simple:** Overusing animations can lead to a cluttered UI. Use them where they add value.
- **Performance:** Avoid complex animations that can lead to performance issues. Use tools like the Flutter DevTools to monitor performance.
- **Consistency:** Maintain a consistent animation style throughout your app for a cohesive user experience.
- **User Control:** Allow users to control certain animations (e.g., play/pause) where appropriate.

There are two type of Widget Lifecycles

- **StatelessWidget Lifecycle**
- **StatefulWidget Lifecycle**

A StatelessWidget is immutable and does not maintain any state. Its lifecycle is straightforward:

- **Construction:** When a StatelessWidget is created, its constructor is called.
- **Build Method:** The build method is called to render the widget. This is where you define the UI.

Syntax:

```
class MyStatelessWidget extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Text('Hello, StatelessWidget!');  
  }  
}
```

1. StatefulWidget Lifecycle

StatefulWidget is mutable and can maintain state across its lifecycle. The lifecycle can be broken down into several key phases:

Creation

- **Constructor:** The constructor of the StatefulWidget is called. This is where you can initialize any final fields and set up initial state.

- **createState():** The createState method is called, which creates the associated State object.

2. State Initialization:

- **initState():** This method is called once when the State object is created. It's a good place to initialize data or set up listeners.
- **build():** After initState, the build method is called to construct the UI based on the current state.

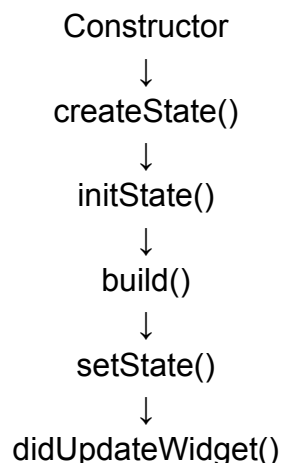
State Changes:

- **setState():** When you call setState, the framework knows that the internal state of the widget has changed. It will schedule a rebuild of the widget by calling the build method again.
- **didUpdateWidget():** This method is called whenever the parent widget rebuilds and the state object needs to be updated with new configurations.

Lifecycle Management:

- **deactivate():** This method is called when the widget is removed from the widget tree but before it is disposed of. It's useful for cleanup before the widget is fully removed.
- **dispose():** This method is called when the State object is permanently removed from the widget tree. It's a good place to release resources, such as controllers or streams.

StatefulWidget Lifecycle Flow is Like,



↓
deactivate()
↓
dispose()

I'll Show you simply with an Example,

```
class CounterWidget extends StatefulWidget {  
  @override  
  _CounterWidgetState createState() => _CounterWidgetState();  
}
```

```
class _CounterWidgetState extends State<CounterWidget> {  
  int _counter = 0;
```

```
  @override  
  void initState() {  
    super.initState();  
    // Initialize state or data here  
  }
```

```
  @override  
  void didUpdateWidget(CounterWidget oldWidget) {  
    super.didUpdateWidget(oldWidget);  
    // Handle updates to the widget here  
  }
```

```
  void _incrementCounter() {  
    setState(() {  
      _counter++;  
    });  
  }
```

```
  @override  
  void dispose() {  
    // Clean up any resources here  
    super.dispose();  
  }
```

```
  @override  
  Widget build(BuildContext context) {  
    return Column(  
      children: [  
        Text('Counter: $_counter'),  
        ElevatedButton(  
          onPressed: _incrementCounter,  
          child: Text('Increment'), ), ], );  
  }
```


In Flutter, widgets can be grouped into multiple categories based on their features

- Platform specific widgets
- Layout widgets
- State maintenance widgets
- Platform independent / basic widgets

1. Platform specific widgets

Flutter has widgets specific to a particular platform - Android or iOS.

- Android specific widgets are designed in accordance with Material design guidelines by Android OS. Android specific widgets are called Material widgets.
- iOS specific widgets are designed in accordance with Human Interface Guidelines by Apple and they are called Cupertino widgets.

Most used material widgets are :

Scaffold
AppBar
BottomNavigationBar
TabBar
TabBarView
ListTile
RaisedButton
FloatingActionButton
FlatButton
IconButton
DropdownButton
PopupMenuButton
AppBar
TextField
Checkbox
Radio
Switch
Slider
Date & Time Pickers
SimpleDialog
AlertDialog

Most used Cupertino widgets are :

CupertinoButton
CupertinoPicker
CupertinoDatePicker
CupertinoTimerPicker
CupertinoNavigationBar
CupertinoTabBar
CupertinoTabScaffold
CupertinoTabView
CupertinoTextField
CupertinoDialog
CupertinoDialogAction
CupertinoFullscreenDialogTransition
CupertinoPageScaffold
CupertinoPageTransition
CupertinoActionSheet
CupertinoActivityIndicator
CupertinoAlertDialog
CupertinoPopupSurface
CupertinoSlider

Layouts

In Flutter, a widget can be created by composing one or more widgets. To compose multiple widgets into a single widget, Flutter provides a large number of widgets with layout features. For example, the child widget can be centered using the Center widget.

Some of the popular layout widgets are as follows:

- **Container:** A rectangular box decorated using BoxDecoration widgets with background, border and shadow.
- **Center:** Center its child widget
- **Row:** Arrange its children in the horizontal direction.
- **Column:** Arrange its children in the vertical direction.
- **Stack:** Arrange one above the another

2. State maintenance widgets

In Flutter, all widgets are either derived from StatelessWidget or StatefulWidget. Widget derived from StatelessWidget does not have any state information but it may contain a widget derived from StatefulWidget. The dynamic nature of the application is through interactive behavior of the widgets and the state changes during interaction. For example, tapping a counter button will increase / decrease the internal state of the counter by one and reactive nature of the Flutter widget will auto re-render the widget using new state information.

3. Platform independent / basic widgets

Flutter provides a large number of basic widgets to create simple as well as complex user interfaces in a platform independent manner. These are some of the basic widgets in this chapter.

Text

Text widget is used to display a piece of string. The style of the string can be set by using style property and TextStyle class.

Syntax:

```
Text('Hello World!', style: TextStyle(fontWeight: FontWeight.bold))
```

Image

Image widget is used to display an image in the application. Image widget provides different constructors to load images from multiple sources

- Image - Generic image loader using ImageProvider
- Image.asset - Load image from flutter project's assets
- Image.file - Load image from system folder
- Image.memory - Load image from memory
- Image.Network - Load image from network

The easiest option to load and display an image in Flutter is by including the image as assets of the application and load it into the widget on demand.

Create a folder, assets in the project folder and place the necessary images.

In the pubspec.yaml file, you can specify assets by including them in the flutter section under assets.

Eg:

flutter:

assets:

- assets/smiley.png

The complete source code of MyHomePage widget of the hello world application below,

Syntax:

```
class MyHomePage extends StatelessWidget {  
  MyHomePage({Key key, this.title}) : super(key: key);  
  final String title;  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text(this.title),  
      ),  
      body: Center(  
        child: Image.asset("assets/smiley.png")  
      ),  
    );  
  }  
}
```

Here inside the flutter app on the center shows image of smiley.png

The most important properties of the Image widget are:

- image, ImageProvider: Actual image to load
- width, double - Width of the image
- height, double - Height of the image
- alignment, AlignmentGeometry - How to align the image within its bounds

Icon

Icon widget is used to display a glyph from a font described in IconData class. The code to load a simple email icon is given

Syntax:

Icon(Icons.email)

Eg:

```
class MyHomePage extends StatelessWidget {
  MyHomePage({Key key, this.title}) : super(key: key);
  final String title;
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(this.title),
      ),
      body: Center(
        child: Icon(Icons.email)
      ),
    );
  }
}
```

Here inside the flutter app on the center shows icon of email.

Lets, Introduce what is Layout widgets

Layouts in Flutter

Flutter provides quite a lot of specially designed widgets like Container, Center, Align, etc., only for the purpose of laying out the user interface. Widgets build by composing other widgets normally use layout widgets.

Layout widgets can be grouped into two distinct category based on its child:

- Widget supporting a single child.
- Widget supporting multiple child.

1. Single Child Widgets

Single child widgets are great options to create high quality widget having single functionality such as button, label, etc.,

The code to create a simple button using Container widget which is a Single Child Widget.

Syntax:

```
Center(  
  child: Container(  
    width: 200,  
    height: 100,  
    color: Colors.blue,  
    child: Center(  
      child: Text(  
        'Hello, Flutter!',  
        style: TextStyle(  
          color: Colors.white,  
          fontSize: 20,  
        ),  
      ),  
    ),  
  ),  
)
```

Let us check some of the most important single child layout widgets provided by Flutter

- **Padding:** Used to arrange its child widget by the given padding. Here, padding can be provided by EdgeInsets class.
- **Align:** Align its child widget within itself using the value of alignment property. The value for alignment property can be provided by the FractionalOffset class. The FractionalOffset class specifies the offsets in terms of a distance from the top left.

Some of the possible values of offsets are as follows:

- `FractionalOffset(1.0, 0.0)` represents the top right.
- `FractionalOffset(0.0, 1.0)` represents the bottom left.

A sample code about offsets

Syntax:

```
Center(  
  child: Container(  
    height: 100.0,  
    width: 100.0,  
    color: Colors.yellow,  
    child: Align(  
      alignment: FractionalOffset(0.2, 0.6),  
      child: Container(  
        height: 40.0,  
        width: 40.0,  
        color: Colors.red,  
      ),  
    ),  
  ),  
)
```

Single child Widgets are categorized so many in types, Such as:

- `SizeBox`
- `DecoratedBox`
- `ClipRect`
- `Transform`
- `FractionallySizeBox`
- `SingleChildScrollView`

2. Multiple Child Widgets

In Flutter, multiple child widgets can hold more than one child widget. These widgets are typically used to create complex layouts by organizing multiple elements together.

Some of the frequently used widgets in Multiple Child Widgets are;

- **Column:** Arranges its children in a vertical line.
- **Row:** Arranges its children in a horizontal line.
- **Stack:** Allows children to be layered on top of each other.

- **ListView**: Creates a scrollable list of widgets.
- **GridView**: Creates a scrollable grid of widgets.
- **Wrap**: Arranges its children in a horizontal or vertical wrap.
- **Flex**: A customizable version of Row and Column, allowing for flexible layouts.
- **PageView**: Creates a scrollable list of pages for swiping between them.
- **ExpansionPanelList**: A scrollable list of expandable panels.

TextFormField

TextFormField is a Flutter widget that allows users to input text. It can be used in forms to collect user data.

There are some important properties in TextFormField class

- **controller**: A TextEditingController that can be used to retrieve the current value of the text field.
- **decoration**: Defines the visual appearance of the input field (e.g., hint text, label text).
- **validator**: A function that validates the input and returns an error message if invalid.
- **onChanged**: A callback function that is called whenever the text changes.
- **obscureText**: A boolean that, when set to true, hides the text (useful for passwords).
- **keyboardType**: Specifies the type of keyboard to display (e.g., TextInputType.emailAddress).

Syntax:

```
class MyForm extends StatelessWidget {
  final TextEditingController _controller = TextEditingController();
```

```
@override
```

```
Widget build(BuildContext context) {
  return Form(
    child: TextFormField(
      controller: _controller,
      decoration: InputDecoration(
        labelText: 'Enter your email',
        hintText: 'example@example.com',
      ),
    ),
    validator: (value) {
      if (value!.isEmpty) {
        return 'Please enter some text';
      }
    }
  );
}
```

```

        return null;
      },
    ),
  );
}
}

```

Button Widgets

In Flutter, Buttons come in various types, each designed for specific functionalities and use cases.

1. FloatingActionButton

A FloatingActionButton is a circular button that floats above the content of the app. It's typically used for a primary action.

Properties

- **onPressed**: Callback function that is triggered when the button is pressed.
- **child**: Typically contains an Icon widget or text.
- **backgroundColor**: Color of the button.
- **tooltip**: Text that describes the action when the button is pressed.

Syntax:

```

FloatingActionButton(
  onPressed: () {
    // Action to perform
  },
  child: Icon(Icons.add),
  tooltip: 'Add Item',
)

```

2. ElevatedButton

An ElevatedButton is a material design button that has elevation and appears raised above the background.

Properties

- **onPressed**: Callback function for button press.
- **child**: Usually contains text or an icon.
- **style**: Customize appearance using ButtonStyle (e.g., padding, shape).

Syntax:

```
ElevatedButton(  
  onPressed: () {  
    // Action to perform  
  },  
  child: Text('Submit'),  
  style: ElevatedButton.styleFrom(  
    padding: EdgeInsets.symmetric(horizontal: 20, vertical: 10),  
  ),  
)
```

3. IconButton

An IconButton is a button that displays an icon.

Properties

- **onPressed**: Callback function when the button is pressed.
- **icon**: The icon displayed within the button.
- **tooltip**: Text that describes the button action.

Syntax:

```
IconButton(  
  onPressed: () {  
    // Action to perform  
  },  
  icon: Icon(Icons.favorite),  
  tooltip: 'Favorite',  
)
```

4. TextButton

A TextButton is a button that displays text with no elevation.

Properties

- **onPressed**: Callback function for button press.
- **child**: Usually contains text.

Syntax:

```
TextButton(  
  onPressed: () {  
    // Action to perform  
  },  
  child: Text('Cancel'),  
)
```

5. OutlinedButton

An OutlinedButton is a button with a border but no fill. It's used for actions that are less important than the primary action.

Properties:

- **onPressed**: Callback function triggered when the button is pressed.
- **child**: Usually contains text or an icon.
- **style**: Customizes the appearance (border, padding).

Syntax:

```
OutlinedButton(  
  onPressed: () {  
    // Action to perform  
  },  
  child: Text('Learn More'),  
)
```

6. MaterialButton

A MaterialButton is a low-level button that can be customized extensively. It is used for creating custom button styles.

Properties:

- **onPressed**: Callback function triggered when the button is pressed.
- **child**: Contains the content of the button (text, icon).
- **color**: Background color of the button.
- **textColor**: Color of the text/icon.

Syntax:

```
MaterialButton(  
  onPressed: () {  
    // Action to perform
```

```

},
child: Text('Custom Button'),
color: Colors.blue,
textColor: Colors.white,
)

```

7. InkWell

InkWell is a Material Design widget that provides a visual reaction (ripple effect) to touches. Inkwell is same as the use of Buttons also,

Properties:

- **onTap**: Callback function triggered when the user taps the widget.
- **child**: The widget that will be wrapped with the ripple effect.
- **splashColor**: Color of the ripple effect.
- **highlightColor**: Color when the widget is pressed.

Syntax:

```

InkWell(
  onTap: () {
    print('InkWell tapped!');
  },
  splashColor: Colors.blue.withOpacity(0.5),
  child: Container(
    padding: EdgeInsets.all(16.0),
    child: Text('Tap Me'),
  ),
)

```

8. Toggle

Toggles allow users to switch between two states, such as on/off.

a. Switch

A Switch widget toggles between on/off states.

Properties

- **value**: A boolean value representing the current state (true for on).
- **onChanged**: A callback that is triggered when the state changes.

Syntax:

```
bool _isSwitched = false;
```

```
Switch(  
  value: _isSwitched,  
  onChanged: (value) {  
    setState(() {  
      _isSwitched = value;  
    });  
  },  
)
```

b. Checkbox

A Checkbox allows users to select or deselect an option.

Properties

- **value:** A boolean value representing the checked/unchecked state.
- **onChanged:** A callback function for state changes.

Syntax:

```
bool _isChecked = false;
```

```
Checkbox(  
  value: _isChecked,  
  onChanged: (value) {  
    setState(() {  
      _isChecked = value!;  
    });  
  },  
)
```

c. ToggleButtons

ToggleButtons are a group of buttons that allow for multiple selections.

Properties

- **isSelected:** A list of boolean values representing the selected states of each button.
- **onPressed:** Callback function triggered when a button is pressed.

Syntax:

```
List<bool> _isSelected = [false, false, false];
```

```
ToggleButtons(  
  children: [Icon(Icons.format_bold), Icon(Icons.format_italic), Icon(Icons.format_underline)],  
  isSelected: _isSelected,  
  onPressed: (index) {  
    setState(() {  
      _isSelected[index] = !_isSelected[index];  
    });  
  },  
)
```

d. Radio Button

A Radio button allows users to select one option from a group.

Properties:

value: The value of the radio button.

groupValue: The value of the currently selected radio button in the group.

onChanged: A callback function that is called when the radio button is selected.

Syntax:

```
int? _selectedValue;
```

```
Row(  
  children: [  
    Radio(  
      value: 1,  
      groupValue: _selectedValue,  
      onChanged: (value) {  
        setState(() {  
          _selectedValue = value;  
        });  
      },  
    ),  
    Text('Option 1'),  
    Radio(  
      value: 2,  
      groupValue: _selectedValue,  
      onChanged: (value) {  
        setState(() {  
          _selectedValue = value;  
        });  
      },  
    ),  
  ],  
)
```

```

    },
  ),
  Text('Option 2'),
],
)

```

Navigation

The Navigator widget is responsible for managing a stack of routes.

Methods

- **push()**

Adds a new route to the stack.

Syntax:

```
push()
```

```
Navigator.push(context,MaterialPageRoute(builder: (context) => SecondPage()),);
```

- **pop()**

Removes the top route from the stack.

Syntax:

```
Navigator.pop(context);
```

- **pushNamed()**

Navigates to a named route.

Syntax:

```
Navigator.pushNamed(context, '/second');
```

To define named routes, you can specify them in the MaterialApp widget.

Syntax:

```

MaterialApp(
  routes: {
    '/': (context) => HomePage(),
    '/second': (context) => SecondPage(),
  },
  initialRoute: '/',
);

```

Validation in Flutter

Validation is a crucial part of form handling in any application. In Flutter, you can validate user input in forms using the `TextFormField` widget, along with the `Form` widget. This allows you to ensure that the data entered by users meets certain criteria before being processed.

Important Concepts in Validation in Flutter

1. `TextFormField`

`TextFormField` is a special type of input field that integrates validation with the `Form` widget. It supports a validator function, which is called when the form is submitted.

2. `Form` Widget

The `Form` widget acts as a container for form fields. It manages the state of the form and provides validation.

3. `GlobalKey`

A `GlobalKey<FormState>` is used to uniquely identify the `Form` and allows you to call methods on it, such as validation.

Implementation Steps

Step 1: Set Up Your Flutter Project

Create a new Flutter project or use an existing one.

Step 2: Create a Form with Validation

Here's a simple example that demonstrates how to create a form with validation:

Syntax:

```
class FeedbackForm extends StatefulWidget {  
  @override  
  _FeedbackFormState createState() => _FeedbackFormState();  
}
```

```
class _FeedbackFormState extends State<FeedbackForm> {  
  final _formKey = GlobalKey<FormState>();  
  String? _username;
```

```
String? _feedback;
```

```
@override
```

```
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(title: Text('Feedback Form')),  
    body: Padding(  
      padding: EdgeInsets.all(16.0),  
      child: Form(  
        key: _formKey,  
        child: Column(  
          children: [  
            // Username Field  
            TextFormField(  
              decoration: InputDecoration(labelText: 'Username'),  
              validator: (value) {  
                if (value == null || value.isEmpty) {  
                  return 'Please enter your username';  
                }  
                return null; // Validation passed  
              },  
              onSaved: (value) => _username = value,  
            ),  
            // Feedback Field  
            TextFormField(  
              decoration: InputDecoration(labelText: 'Feedback'),  
              maxLines: 3,  
              validator: (value) {  
                if (value == null || value.isEmpty) {  
                  return 'Please provide your feedback';  
                }  
                return null; // Validation passed  
              },  
              onSaved: (value) => _feedback = value,  
            ),  
            SizedBox(height: 20),  
            // Submit Button  
            ElevatedButton(  
              onPressed: () {  
                if (_formKey.currentState!.validate()) {  
                  _formKey.currentState!.save(); // Save the input values  
                  // Process the data (e.g., send to a server)  
                  ScaffoldMessenger.of(context).showSnackBar(  
                    SnackBar(content: Text('Thank you, $_username!')),  
                  );  
                }  
              },  
              child: Text('Submit'),  
            ),  
          ],  
        ),  
      ),  
    ),  
  );  
}
```



```

    ],
  ),
),
),
);
}
}

```

Explanation of the Code

- **GlobalKey**: We create a `GlobalKey<FormState>` to manage the form state.
- **TextFormField**: Each field has a validator function to check for validity:
 - For the email field, it checks if the input is empty or if the email format is valid.
 - For the password field, it checks if the input is empty or if the password length is less than 6 characters.
- **onSaved**: If validation passes, the values are saved in the corresponding variables.
- **Submit Button**: When the button is pressed, it validates the form. If validation passes, it saves the data and can perform further actions (like sending data to a server).

Common Validation Patterns

1. **Required Fields**: Always check if the field is empty.
2. **Email Validation**: Use regex to validate email formats.
3. **Password Strength**: Check the length and possibly the complexity of passwords.
4. **Custom Validation**: You can create custom validation rules for specific use cases.
5. **Asynchronous Validation**: For cases like checking if an email already exists, you can use asynchronous functions in the validator.

Stack Widgets

A Stack widget allows you to overlay multiple widgets on top of each other.

Properties:

- **children:** A list of widgets to be stacked.
- **alignment:** Aligns children within the stack.

Syntax:

```
Stack(  
  alignment: Alignment.center,  
  children: [  
    Container(width: 100, height: 100, color: Colors.red),  
    Container(width: 80, height: 80, color: Colors.blue),  
    Container(width: 60, height: 60, color: Colors.green),  
  ],  
)
```

ListView in Flutter

1. ListView

A ListView is a scrollable list of widgets.

Syntax:

```
ListView(  
  children: <Widget>[  
    ListTile(title: Text('Item 1')),  
    ListTile(title: Text('Item 2')),  
    ListTile(title: Text('Item 3')),  
  ],  
)
```

2. ListView.builder

ListView.builder is used to create a scrollable list with a builder pattern, making it efficient for long or infinite lists.

Properties:

itemCount: Number of items in the list.

itemBuilder: Function that builds each item.

Syntax:

```
ListView.builder(  
  itemCount: 100,  
  itemBuilder: (context, index) {  
    return ListTile(title: Text('Item ${index + 1}'));  
  },  
)
```

3. **ListView.separated**

ListView.separated is similar to ListView.builder but allows you to add a separator between items.

Properties:

- **itemCount**: Number of items in the list.
- **itemBuilder**: Function that builds each item.
- **separatorBuilder**: Function that builds the separator widget.

Syntax:

```
ListView.separated(  
  itemCount: 100,  
  itemBuilder: (context, index) {  
    return ListTile(title: Text('Item ${index + 1}'));  
  },  
  separatorBuilder: (context, index) => Divider(),  
)
```

CircleAvatar

CircleAvatar is a circular widget that typically displays a user's profile picture or initials.

Properties:

- **backgroundColor**: The background color of the circle.
- **foregroundImage**: An image to display inside the circle.
- **child**: Usually contains text (like initials).
- **radius**: The radius of the circle.

Syntax:

```
CircleAvatar(  
  backgroundColor: Colors.blue,  
  radius: 30,
```

```
child: Text('AB', style: TextStyle(color: Colors.white, fontSize: 20)),  
)
```

Divider

Divider is a thin horizontal line that divides content in the UI.

Properties:

- **height**: The height of the divider.
- **thickness**: The thickness of the line.
- **color**: The color of the line.
- **indent**: The space to the leading edge of the divider.
- **endIndent**: The space to the trailing edge of the divider.

Syntax:

```
Divider(  
  height: 20,  
  thickness: 2,  
  color: Colors.grey,  
  indent: 10,  
  endIndent: 10,  
)
```

Flutter Toast and Snackbar

Flutter Toast

A toast is a message that pops up on the screen for a brief period. While Flutter does not have built-in support for toasts, you can use the `fluttersnackbar` package.

Syntax:

```
Fluttertoast.showToast(  
  msg: "This is a toast message",  
  toastLength: Toast.LENGTH_SHORT,  
  gravity: ToastGravity.BOTTOM,  
  timeInSecForIosWeb: 1,  
  backgroundColor: Colors.black,  
  textColor: Colors.white,  
);
```

Snackbar

A Snackbar is a lightweight message that provides brief feedback about an operation.

Syntax:

```
ScaffoldMessenger.of(context).showSnackBar(  
  SnackBar(  
    content: Text('This is a Snackbar!'),  
    action: SnackBarAction(  
      label: 'Undo',  
      onPressed: () {  
        // Code to execute when the user presses the button  
      },  
    ),  
  ),  
);
```

Date Picker and Time Picker

1. DatePicker

A date picker allows users to select a date.

Syntax:

```
Future<void> _selectDate(BuildContext context) async {  
  final DateTime? picked = await showDatePicker(  
    context: context,  
    initialDate: DateTime.now(),  
    firstDate: DateTime(2000),  
    lastDate: DateTime(2101),  
  );  
  if (picked != null && picked != DateTime.now())  
    setState(() {  
      // Use picked date  
    });  
}
```

2. TimePicker

A time picker allows users to select a time.

Syntax:

```
Future<void> _selectTime(BuildContext context) async {  
  final TimeOfDay? picked = await showTimePicker(  
    context: context,  
    initialTime: TimeOfDay.now(),  
  );  
}
```

```

);
if (picked != null) {
  setState(() {
    // Use picked time
  });
}
}
}

```

AlertDialog Box

An AlertDialog is a modal dialog that provides important information or asks for confirmation.

Syntax:

```

showDialog(
  context: context,
  builder: (BuildContext context) {
    return AlertDialog(
      title: Text('Alert'),
      content: Text('This is an alert dialog.'),
      actions: [
        TextButton(
          child: Text('Cancel'),
          onPressed: () {
            Navigator.of(context).pop();
          },
        ),
        TextButton(
          child: Text('OK'),
          onPressed: () {
            // Perform action
            Navigator.of(context).pop();
          },
        ),
      ],
    );
  },
);

```

Image Picker

The ImagePicker allows users to select images from the gallery or take a photo using the camera. You can use the image_picker package for this functionality.

Syntax:

```

final ImagePicker _picker = ImagePicker();

Future<void> _pickImage() async {
  final XFile? image = await _picker.pickImage(source: ImageSource.gallery);
  if (image != null) {
    // Use the selected image
  }
}

```

NavigationBar

A `NavigationBar` provides a bottom navigation interface to switch between different views or pages within an application.

Syntax:

```

class BottomNavBarExample extends StatefulWidget {
  @override
  _BottomNavBarExampleState createState() => _BottomNavBarExampleState();
}

```

```

class _BottomNavBarExampleState extends State<BottomNavBarExample> {
  int _selectedIndex = 0;

```

```

  final List<Widget> _widgetOptions = <Widget>[
    Text('Home'),
    Text('Search'),
    Text('Profile'),
  ];

```

```

  void _onItemTapped(int index) {
    setState(() {
      _selectedIndex = index;
    });
  }

```

```

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Navigation Bar Example')),
      body: Center(child: _widgetOptions.elementAt(_selectedIndex)),
      bottomNavigationBar: BottomNavigationBar(
        items: const <BottomNavigationBarItem>[
          BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Home'),
          BottomNavigationBarItem(icon: Icon(Icons.search), label: 'Search'),
          BottomNavigationBarItem(icon: Icon(Icons.person), label: 'Profile'),
        ],
        currentIndex: _selectedIndex,

```

```

        onTap: _onItemTapped,
      ),
    );
  }
}

```

Drawer

A Drawer is a sliding panel that provides a navigation menu, typically accessed by swiping from the side or tapping an icon.

Syntax:

```

class DrawerExample extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Drawer Example')),
      drawer: Drawer(
        child: ListView(
          padding: EdgeInsets.zero,
          children: <Widget>[
            DrawerHeader(
              child: Text('Drawer Header', style: TextStyle(color: Colors.white, fontSize: 24)),
              decoration: BoxDecoration(color: Colors.blue),
            ),
            ListTile(
              leading: Icon(Icons.home),
              title: Text('Home'),
              onTap: () {
                // Navigate to home
                Navigator.pop(context);
              },
            ),
            ListTile(
              leading: Icon(Icons.settings),
              title: Text('Settings'),
              onTap: () {
                // Navigate to settings
                Navigator.pop(context);
              },
            ),
          ],
        ),
      ),
      body: Center(child: Text('Swipe from the left or tap the top-left icon')),
    );
  }
}

```


TabBar

A TabBar allows users to switch between different views or pages using tabs.

Syntax:

```
class TabBarExample extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return DefaultTabController(  
      length: 3,  
      child: Scaffold(  
        appBar: AppBar(  
          title: Text('Tab Bar Example'),  
          bottom: TabBar(  
            tabs: [  
              Tab(icon: Icon(Icons.home), text: 'Home'),  
              Tab(icon: Icon(Icons.search), text: 'Search'),  
              Tab(icon: Icon(Icons.person), text: 'Profile'),  
            ],  
          ),  
        ),  
        body: TabBarView(  
          children: [  
            Center(child: Text('Home')),  
            Center(child: Text('Search')),  
            Center(child: Text('Profile')),  
          ],  
        ),  
      ),  
    );  
  }  
}
```

State management is the most essential factor for a Flutter developer, as it ensures applications respond effectively to user interactions and maintain a seamless experience. Mastering it is key to building efficient and scalable apps.

State Management

State management in Flutter is crucial for managing the flow of data and ensuring that the UI reflects changes in the underlying data.

There are so many State Managements in Flutter, They are;

- **Provider**
- **GetX**
- **Riverpod**
- **BLoC, Cubit**
- **GetIt**
- **Redux**
- **MobX**

We were Discuss about Two popular state management solutions in Flutter are Provider and GetX.

Provider

Provider is a wrapper around InheritedWidget, making it easier to manage state and listen to changes in the state. It's part of the Flutter community and is widely used for its simplicity and effectiveness.

Features of Provider

- **Scoped Access:** Allows you to provide state to specific parts of the widget tree.
- **ChangeNotifier:** A class that can notify listeners when the state changes.
- **Ease of Testing:** Since the state is separated from the UI, it's easier to write unit tests.

Let's learn How to Use Provider:

Step 1:

Add Provider to pubspec.yaml:
dependencies:
provider: ^latest_version

Step 2. Create a ChangeNotifier: create a file counter_controller.dart inside the file;

```
class Counter extends ChangeNotifier {
  int _count = 0;

  int get count => _count;

  void increment() {
    _count++;
    notifyListeners(); // Notify listeners of the change
  }
}
```

Step 3:

Provide the ChangeNotifier:

```
void main() {  
  runApp(  
    ChangeNotifierProvider(  
      create: (context) => Counter(),  
      child: MyApp(),  
    ),  
  );  
}
```

Step 4:

Consume the Provider

```
class CounterWidget extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    final counter = Provider.of<Counter>(context);  
    return Text('${counter.count}');  
  }  
}
```

Here is how we use Provider State management.

GetX

GetX is a powerful and lightweight solution for state management, dependency injection, and routing. It offers a reactive programming model that simplifies state management.

Key Features:

- **Reactive Programming:** Automatically updates the UI when the state changes.
- **Dependency Injection:** Simplifies the management of dependencies within your app.
- **Routing:** Integrated routing system.
- **Performance:** Minimal boilerplate and efficient memory usage.

How we can Use GetX:

Step 1:

Add GetX to pubspec.yaml:

dependencies:
get: ^latest_version

Step 2:

Create a Controller:

```
class CounterController extends GetxController {  
  var count = 0.obs; // Observable variable  
  
  void increment() {  
    count++; // Automatically triggers UI updates  
  }  
}
```

Step 3:

Use the Controller:

```
void main() {  
  Get.put(CounterController()); // Initialize the controller  
  runApp(MyApp());  
}
```

Step 4:

Consume the GetX to UI:

```
class CounterWidget extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    final CounterController controller = Get.find(); // Access the controller  
    return Obx(() => Text('${controller.count}')); // Automatically updates  
  }  
}
```

Here is a comparison between both Provider and GetX

Feature	Provider	GetX
Feature	Moderate	GetX
Learning Curve	Good	Easy
Performance	Slightly Verbose	Excellent
Code Readability	Easy	Concise
Testing	to Test	Also easy
Routing	No built-in support	Integrated routing
Dependency Injection	Not built-in	Built-in

Both Provider and GetX are excellent choices for state management in Flutter. Provider is great for simpler applications or when you want to follow Flutter's reactive programming model closely. GetX, on the other hand, is more powerful with built-in routing and dependency injection, making it suitable for larger applications that require more features and performance optimizations.

Architectures Design Patterns in Flutter

In today's development landscape, understanding architectural design patterns is crucial for creating reliable, high-performance code, and mastering these patterns can significantly enhance coding stability and maintainability.

Flutter offers several architectural patterns that help in organizing code, enhancing maintainability, and improving performance.

Some of the most popular architectures used in Flutter are,

1. MVVM (Model-View-ViewModel):

- **Model:** Data and business logic.
- **View:** UI components that present data.
- **ViewModel:** Acts as a bridge between the Model and View, providing data and handling user interactions.

2. MVC (Model-View-Controller):

- **Model:** Represents the data and business logic.
- **View:** UI components that display data.
- **Controller:** Manages user input and updates the model and view.

3. Clean Architecture:

- Emphasizes separation of concerns and testability.
- Divides the application into layers:
 - Presentation
 - Domain
 - Data.
- Each layer has a specific responsibility, promoting high cohesion and low coupling.

Here, We need to explore the MVVM and MVC architecture patterns to enhance performance and improve our development practices.

MVVM Architecture

MVVM (Model-View-ViewModel) is a software architectural pattern that facilitates the separation of the user interface (UI) from the business logic. In Flutter, MVVM helps in creating a clean, manageable, and testable codebase.

Components of MVVM

MVVM pattern consists of three main components:

1. Model

- Represents the data and business logic of the application.
- Responsible for fetching data from APIs or databases.
- Notifies the ViewModel when data changes.

2. View

- Represents the UI of the application.
- Displays data and handles user interactions.
- Updates in response to changes in the ViewModel.

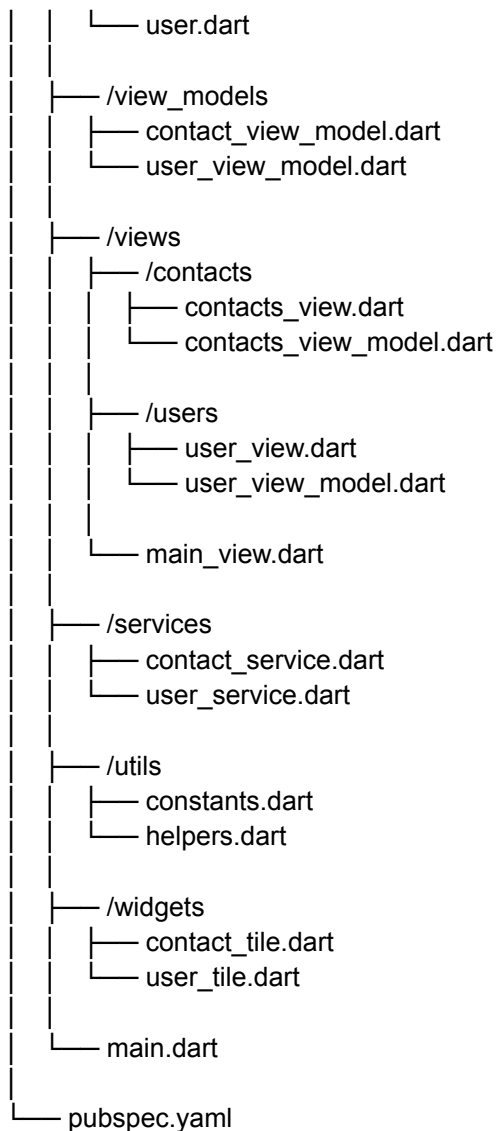
3. ViewModel

- Acts as a bridge between the Model and the View.
- Holds the data to be displayed and handles the UI logic.
- Exposes data in a way that the View can easily consume.
- Notifies the View about data changes, usually through a reactive mechanism.

Creating a well-organized folder structure for an MVVM application can greatly enhance maintainability and scalability. Below is a suggested folder structure that you can adapt based on your project's needs.

We'll will Show you an Example of Folder Structure with MVVM Architecture

```
/your_flutter_app
|
|— /lib
|   |— /models
|   |   |— contact.dart
```



Here is an example of how to implement the MVVM (Model-View-ViewModel) architecture in Flutter, showcasing a structured approach to separating the UI from business logic, which enhances code maintainability and testability.

Setting Up a Flutter Project

To demonstrate MVVM in Flutter, let's create a simple application that fetches and displays a list of items from a mock API.

Step 1: Create a Flutter Project

- 1.flutter create mvvm_example
- 2.cd mvvm_example
- 3.code .

Step 2: Add Dependencies

Open pubspec.yaml and add the following dependencies:(We use Provider as State management here so add provider package also)

dependencies:

```
flutter:  
  sdk: flutter  
http: ^0.13.3  
provider: ^6.0.0
```

Implementing MVVM in Flutter

1. Model

Create a model class to represent the data structure. In this example, we'll create a Item model.

```
lib/models/item.dart:  
class Item {  
  final int id;  
  final String title;  
  
  Item({required this.id, required this.title});  
  
  factory Item.fromJson(Map<String, dynamic> json) {  
    return Item(  
      id: json['id'],  
      title: json['title'],  
    );  
  }  
}
```

2. View

Create the UI that will consume the ViewModel.

```
lib/main.dart:  
import 'package:flutter/material.dart';  
import 'package:provider/provider.dart';  
import 'view_models/item_view_model.dart';  
  
void main() {  
  runApp(  
    ChangeNotifierProvider(  
      create: (context) => ItemViewModel(),  
      child: MaterialApp(  
        title: 'Flutter Demo',  
        theme: ThemeData(  
          primaryColor: Colors.pink,  
        ),  
        home: ItemView(),  
      ),  
    ),  
  );  
}
```



```

        create: (context) => ItemViewModel(),
        child: MyApp(),
      ),
    );
  }

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'MVVM Example',
      home: ItemListView(),
    );
  }
}

class ItemListView extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    final viewModel = Provider.of<ItemViewModel>(context);

    return Scaffold(
      appBar: AppBar(title: Text('Items')),
      body: viewModel.isLoading
        ? Center(child: CircularProgressIndicator())
        : ListView.builder(
            itemCount: viewModel.items.length,
            itemBuilder: (context, index) {
              final item = viewModel.items[index];
              return ListTile(
                title: Text(item.title),
              );
            },
          ),
    );
  }
}

```

3. ViewModel

Create a ViewModel that will handle the fetching of data and expose it to the View.

```

lib/view_models/item_view_model.dart:
import 'dart:convert';
import 'package:http/http.dart' as http;
import 'package:flutter/material.dart';
import '../models/item.dart';

class ItemViewModel extends ChangeNotifier {

```

```

List<Item> _items = [];
bool _isLoading = true;

List<Item> get items => _items;
bool get isLoading => _isLoading;

ItemViewModel() {
  fetchItems();
}
Future<void> fetchItems() async {
  final response = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts'));

  if (response.statusCode == 200) {
    final List<dynamic> data = json.decode(response.body);
    _items = data.map((json) => Item.fromJson(json)).toList();
    _isLoading = false;
    notifyListeners();
  } else {
    throw Exception('Failed to load items');
  }
}
}

```

We should also learn about the MVC (Model-View-Controller) architecture, as it is another important design pattern. Understanding MVC will provide us with valuable insights into structuring our applications effectively and help us make informed decisions about which architecture to use in different scenarios.

MVC Architecture in Flutter

MVC, which stands for Model-View-Controller, is a design pattern used in software development to separate concerns. This helps organize code in a way that makes applications easier to manage, test, and scale. Each component has its own responsibility.

Components of MVC

MVC pattern consists of three main components:

- **Model:** Represents the data and business logic.
- **View:** The user interface that displays data.
- **Controller:** Handles input, interacts with the model, and updates the view.

Benefits of MVC

Separation of Concerns: Each component has a distinct responsibility.

Reusability: Models and Views can be reused across different parts of the application.

Maintainability: Easier to maintain and update the code.

Testability: Each component can be tested independently.

Folder Structure

Here's an Example of Folder Structure with MVVM Architecture

```
lib/
├── models/
│   └── user.dart      // User model definition
├── views/
│   └── user_view.dart // User interface for displaying user info
├── controllers/
│   └── user_controller.dart // Logic to manage user data
└── main.dart          // Main entry point of the application
```

Here is an example of how to implement the MVC (Model-View-Controller) architecture in Flutter. While we used Provider for state management in the previous MVVM architecture, in this case, we will utilize GetX to handle state management more efficiently.

Implementing MVVM in Flutter

First, you need to create a new Flutter project and add the necessary dependencies, just as we did in the previous session and change the provider to get package also.

1. Model

Create a file named user.dart in the models folder:

```
// lib/models/user.dart
class User {
  final int id;
  final String name;
  final String email;

  User({required this.id, required this.name, required this.email});
```

```

factory User.fromJson(Map<String, dynamic> json) {
  return User(
    id: json['id'],
    name: json['name'],
    email: json['email'],
  );
}
}

```

2. View

Create a file named `user_list_view.dart` in the views folder:

```

// lib/views/user_list_view.dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';
import '../controllers/user_controller.dart';

class UserListView extends StatelessWidget {
  final UserController controller = Get.put(UserController());

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('User List')),
      body: Obx(() {
        if (controller.isLoading.value) {
          return Center(child: CircularProgressIndicator());
        }

        return ListView.builder(
          itemCount: controller.users.length,
          itemBuilder: (context, index) {
            final user = controller.users[index];
            return ListTile(
              title: Text(user.name),
              subtitle: Text(user.email),
            );
          },
        );
      }));
  }
}

```

3. Controller

Create a file named `user_controller.dart` in the controllers folder:

```
// lib/controllers/user_controller.dart
import 'package:get/get.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';
import '../models/user.dart';

class UserController extends GetxController {
  var users = <User>[].obs; // Observable list of users
  var isLoading = true.obs; // Observable loading state

  @override
  void onInit() {
    fetchUsers();
    super.onInit();
  }

  void fetchUsers() async {
    try {
      isLoading(true);
      final response = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/users'));

      if (response.statusCode == 200) {
        var jsonData = json.decode(response.body);
        users.value = (jsonData as List).map((user) => User.fromJson(user)).toList();
      }
    } finally {
      isLoading(false);
    }
  }
}
```

into the `main.dart` page contains,

```
// lib/main.dart
import 'package:flutter/material.dart';
import 'views/user_list_view.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
```

```

return MaterialApp(
  title: 'User List App',
  theme: ThemeData(primarySwatch: Colors.blue),
  home: UserListView(),
);
}
}

```

Shared Preferences

Shared Preferences is a way to store simple data persistently in key-value pairs. It's commonly used for saving user settings or app preferences.

Implementation Steps

- **Add Dependency:** Include the shared_preferences package in your pubspec.yaml: shared_preferences: ^2.0.0.
- **Import Package:** import 'package:shared_preferences/shared_preferences.dart';

Syntax:

```

class SharedPrefExample extends StatefulWidget {
  @override
  _SharedPrefExampleState createState() => _SharedPrefExampleState();
}

```

```

class _SharedPrefExampleState extends State<SharedPrefExample> {
  String _storedValue = "";

```

```

  @override
  void initState() {
    super.initState();
    _loadData();
  }

```

```

  _loadData() async {
    SharedPreferences prefs = await SharedPreferences.getInstance();
    setState(() {
      _storedValue = prefs.getString('key') ?? "";
    });
  }

```

```

  _saveData(String value) async {
    SharedPreferences prefs = await SharedPreferences.getInstance();
    prefs.setString('key', value);
  }

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text('Shared Preferences Example')),
    body: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Text('Stored Value: $_storedValue'),
        TextField(
          onSubmitted: (value) {
            _saveData(value);
            _loadData();
          },
          decoration: InputDecoration(labelText: 'Enter Value'),
        ), ], ), );}}

```

creating a simple Flutter project that implements CRUD (Create, Read, Update, Delete) operations using Shared Preferences, following the MVVM (Model-View-ViewModel) architecture.

Project Structure

```

my_shared_preferences_mvvm/
├── android/
├── ios/
├── lib/
│   ├── main.dart
│   ├── models/
│   │   └── item_model.dart
│   ├── services/
│   │   └── shared_preferences_service.dart
│   ├── viewmodels/
│   │   └── item_viewmodel.dart
│   └── views/
│       └── home_page.dart
└── pubspec.yaml
...

```

Step 1: pubspec.yaml

Ensure you add the shared_preferences dependency in your pubspec.yaml file:

dependencies:

flutter:

 sdk: flutter

shared_preferences: ^2.0.10

Step 2: Create a model class to represent your items:

```
lib/models/item_model.dart
class ItemModel {
  final String id;
  final String name;

  ItemModel({required this.id, required this.name});

  Map<String, dynamic> toMap() {
    return {
      'id': id,
      'name': name,
    };
  }
}
static ItemModel fromMap(Map<String, dynamic> map) {
  return ItemModel(
    id: map['id'],
    name: map['name'],
  );
}
```

Step 3: Create a service class to manage Shared Preferences operations:

```
lib/services/shared_preferences_service.dart
import 'dart:convert';
import 'package:shared_preferences/shared_preferences.dart';
import '../models/item_model.dart';

class SharedPreferencesService {
  static const String _itemListKey = 'item_list';

  Future<List<ItemModel>> getItems() async {
    final prefs = await SharedPreferences.getInstance();
    final String? itemListJson = prefs.getString(_itemListKey);
    if (itemListJson == null) return [];

    final List<dynamic> itemList = json.decode(itemListJson);
    return itemList.map((item) => ItemModel.fromMap(item)).toList();
  }

  Future<void> addItem(ItemModel item) async {
    final prefs = await SharedPreferences.getInstance();
    final List<ItemModel> currentItems = await getItems();
    currentItems.add(item);
    await prefs.setString(_itemListKey, json.encode(currentItems.map((item) => item.toMap()).toList()));
  }

  Future<void> updateItem(ItemModel updatedItem) async {
    final prefs = await SharedPreferences.getInstance();
  }
```



```

    final List<ItemModel> currentItems = await getItems();
    final index = currentItems.indexWhere((item) => item.id == updatedItem.id);
    if (index != -1) {
      currentItems[index] = updatedItem;
      await prefs.setString(_itemListKey, json.encode(currentItems.map((item) => item.toMap()).toList()));
    }
  }
}

Future<void> deleteItem(String id) async {
  final prefs = await SharedPreferences.getInstance();
  final List<ItemModel> currentItems = await getItems();
  currentItems.removeWhere((item) => item.id == id);
  await prefs.setString(_itemListKey, json.encode(currentItems.map((item) => item.toMap()).toList()));
}
}

```

Step 4: Create a ViewModel class to manage the data and handle business logic:

```

lib/viewmodels/item_viewmodel.dart
import 'package:flutter/material.dart';
import '../models/item_model.dart';
import '../services/shared_preferences_service.dart';

class ItemViewModel extends ChangeNotifier {
  final SharedPreferencesService _prefsService = SharedPreferencesService();
  List<ItemModel> items = [];

  Future<void> loadItems() async {
    items = await _prefsService.getItems();
    notifyListeners();
  }

  Future<void> addItem(String name) async {
    final newItem = ItemModel(id: DateTime.now().toString(), name: name);
    await _prefsService.addItem(newItem);
    await loadItems();
  }

  Future<void> updateItem(String id, String name) async {
    final updatedItem = ItemModel(id: id, name: name);
    await _prefsService.updateItem(updatedItem);
    await loadItems();
  }

  Future<void> deleteItem(String id) async {
    await _prefsService.deleteItem(id);
    await loadItems();
  }
}

```

Step 5: Create the main view where users can interact with the app:

```
lib/views/home_page.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../viewmodels/item_viewmodel.dart';
import '../models/item_model.dart';

class HomePage extends StatelessWidget {
  final TextEditingController _controller = TextEditingController();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('CRUD with Shared Preferences (MVVM)'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          children: [
            TextField(
              controller: _controller,
              decoration: InputDecoration(labelText: 'Item Name'),
            ),
            SizedBox(height: 10),
            ElevatedButton(
              onPressed: () {
                final viewModel = Provider.of<ItemViewModel>(context, listen: false);
                viewModel.addItem(_controller.text);
                _controller.clear();
              },
              child: Text('Add Item'),
            ),
            Expanded(
              child: Consumer<ItemViewModel>(
                builder: (context, viewModel, child) {
                  return ListView.builder(
                    itemCount: viewModel.items.length,
                    itemBuilder: (context, index) {
                      final item = viewModel.items[index];
                      return ListTile(
                        title: Text(item.name),
                        trailing: Row(
                          mainAxisAlignment: MainAxisAlignment.min,
                          children: [
                            IconButton(
                              icon: Icon(Icons.edit),
```

```

onPressed: () {
  _controller.text = item.name; // Pre-fill the text field for editing
  showDialog(
    context: context,
    builder: (context) {
      return AlertDialog(
        title: Text('Edit Item'),
        content: TextField(
          controller: _controller,
          decoration: InputDecoration(labelText: 'Item Name'),
        ),
        actions: [
          TextButton(
            onPressed: () {
              viewModel.updateItem(item.id, _controller.text);
              _controller.clear();
              Navigator.of(context).pop();
            },
            child: Text('Save'),
          ),
        ],
      );
    },
  );
),
IconButton(
  icon: Icon(Icons.delete),
  onPressed: () => viewModel.deleteItem(item.id),
), ], ), );
},
),
),
],
),
),
);
}
}

```

Step 6: Set up the entry point of the application and the Provider for state management:

```

lib/main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'views/home_page.dart';
import 'viewmodels/item_viewmodel.dart';

```

```

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (context) => ItemViewModel()..loadItems(),
      child: MaterialApp(
        title: 'CRUD with Shared Preferences (MVVM)',
        theme: ThemeData(
          primarySwatch: Colors.blue,
        ),
        home: HomePage(),
      ),
    );
  }
}

```

There is another local storage option in Flutter called SQLite, which is more reliable than Flutter's Shared Preferences.

SQLite

SQLite is a self-contained, serverless, zero-configuration, and transactional SQL database engine. It is one of the most widely used databases in the world, primarily because of its simplicity, lightweight nature, and full feature set.

Features of SQLite

- **Serverless:** SQLite does not require a separate server process. It operates as a library that gets integrated into applications. This makes it easy to deploy and use.
- **Zero Configuration:** There's no setup or administration required. You can create a database simply by writing to a file.
- **Cross-Platform:** SQLite works on various platforms, including Windows, macOS, Linux, Android, and iOS, making it versatile for developers.
- **Lightweight:** The entire SQLite library is typically less than 500 KiB in size, making it a great choice for mobile applications or devices with limited resources.

Use of SQLite

- **Mobile Applications:** It's commonly used in mobile apps (like Android and iOS) for local data storage because of its lightweight nature.
- **Embedded Systems:** SQLite is often found in embedded devices and applications where resources are limited.
- **Web Browsers:** Many web browsers (e.g., Firefox, Chrome) use SQLite for storing user data, settings, and session information.
- **Testing and Prototyping:** Developers frequently use SQLite for testing and prototyping because it is quick to set up and modify.
- **Data Analysis:** Data analysts use SQLite to manipulate and query small to medium-sized datasets without the overhead of a full database server.

Advantages of Using SQLite

- **Simplicity:** Easy to set up and use, making it accessible for beginners and quick development cycles.
- **Performance:** Fast for read operations and efficient for write operations in many scenarios.
- **Reliability:** Well-tested and widely used, ensuring a high degree of reliability and stability.

Disadvantages of Using SQLite

- **Concurrency Limitations:** While it supports multiple reads, it can handle limited concurrent writes, which may be a concern in high-traffic applications.
- **Data Size Limitations:** Suitable for small to medium datasets but may not be ideal for very large datasets.
- **No User Management:** SQLite does not include user management features. It relies on the host application to manage access and permissions.

Here is an example of how to implement SQLite in a Flutter application and how to integrate SQLite into a Flutter application, This will cover setup, CRUD operations, and best practices.

We should start our app using the MVVM pattern; here is a folder structure to facilitate learning.

```
my_flutter_app/
├── android/
├── ios/
├── lib/
```

```

|   |   |— models/
|   |   |   |— item_model.dart
|   |   |— viewmodels/
|   |   |   |— item_viewmodel.dart
|   |   |— views/
|   |   |   |— my_home_page.dart
|   |   |— database/
|   |   |   |— database_helper.dart
|   |   |— main.dart
|— pubspec.yaml

```

Step 1:

First, you need to add the sqflite and path packages to your pubspec.yaml file dependencies:

```

flutter:
  sdk: flutter
  sqflite: ^2.0.0+4
  path: ^1.8.0

```

Step 2:

lib/models/item_model.dart

```

class ItemModel {
  final int? id;
  final String name;
  final int value;

```

```

  ItemModel({this.id, required this.name, required this.value});

```

```

  Map<String, dynamic> toMap() {
    return {
      'id': id,
      'name': name,
      'value': value,
    };
  }

```

```

  static ItemModel fromMap(Map<String, dynamic> map) {
    return ItemModel(
      id: map['id'],
      name: map['name'],
      value: map['value'],
    );
  }
}

```

Step 3:

```
lib/database/database_helper.dart
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';
import '../models/item_model.dart';

class DatabaseHelper {
  static final DatabaseHelper _instance = DatabaseHelper._internal();
  factory DatabaseHelper() => _instance;

  static Database? _database;

  DatabaseHelper._internal();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB();
    return _database!;
  }

  Future<Database> _initDB() async {
    String path = join(await getDatabasesPath(), 'my_database.db');
    return await openDatabase(
      path,
      version: 1,
      onCreate: _createDB,
    );
  }

  Future _createDB(Database db, int version) async {
    await db.execute("""
      CREATE TABLE items(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT,
        value INTEGER
      )
    """);
  }

  Future<int> insertItem(ItemModel item) async {
    final db = await database;
    return await db.insert('items', item.toMap());
  }

  Future<List<ItemModel>> getItems() async {
    final db = await database;
    final List<Map<String, dynamic>> result = await db.query('items');
    return result.map((item) => ItemModel.fromMap(item)).toList();
  }
}
```

```

}

Future<ItemModel?> getItem(int id) async {
  final db = await database;
  final List<Map<String, dynamic>> result = await db.query(
    'items',
    where: 'id = ?',
    whereArgs: [id],
  );
  return result.isNotEmpty ? ItemModel.fromMap(result.first) : null;
}

Future<int> updateItem(ItemModel item) async {
  final db = await database;
  return await db.update(
    'items',
    item.toMap(),
    where: 'id = ?',
    whereArgs: [item.id],
  );
}

Future<int> deleteItem(int id) async {
  final db = await database;
  return await db.delete(
    'items',
    where: 'id = ?',
    whereArgs: [id],
  );
}
}

```

Step 4:

```

lib/viewmodels/item_viewmodel.dart
import 'package:flutter/material.dart';
import '../database/database_helper.dart';
import '../models/item_model.dart';

class ItemViewModel extends ChangeNotifier {
  final DatabaseHelper _dbHelper = DatabaseHelper();
  List<ItemModel> items = [];

  Future<void> fetchItems() async {
    items = await _dbHelper.getItems();
    notifyListeners();
  }

  Future<void> addItem(ItemModel item) async {

```



```

    await _dbHelper.insertItem(item);
    await fetchItems();
  }

Future<void> deleteItem(int id) async {
  await _dbHelper.deleteItem(id);
  await fetchItems();
}

Future<ItemModel?> getItem(int id) async {
  return await _dbHelper.getItem(id);
}
}

```

Step 5:

```

lib/views/my_home_page.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../viewmodels/item_viewmodel.dart';
import '../models/item_model.dart';

class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('SQLite MVVM Demo'),
      ),
      body: Consumer<ItemViewModel>(
        builder: (context, viewModel, child) {
          return ListView.builder(
            itemCount: viewModel.items.length,
            itemBuilder: (context, index) {
              final item = viewModel.items[index];
              return ListTile(
                title: Text(item.name),
                subtitle: Text('Value: ${item.value}'),
              );
            },
          );
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          // Example of adding an item
          final newItem = ItemModel(name: 'Item ${DateTime.now()}', value: 100);
          Provider.of<ItemViewModel>(context, listen: false).addItem(newItem);
        },
      ),
    );
  }
}

```

```

        child: Icon(Icons.add),
      ),
    );
  }
}

```

Step 6:

lib/main.dart

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'views/my_home_page.dart';
import 'viewmodels/item_viewmodel.dart';

```

```

void main() {
  runApp(MyApp());
}

```

```

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (context) => ItemViewModel()..fetchItems(),
      child: MaterialApp(
        title: 'SQLite MVVM Demo',
        home: MyHomePage(),
      ),
    );
  }
}

```

URL Launcher

The url_launcher package is used to launch a URL in a mobile platform. It can open web URLs, email clients, phone calls, and more.

Implementation Steps

- **Add Dependency:** Include the url_launcher package in your pubspec.yaml:
url_launcher: ^6.0.0
- **Import Package:** import 'package:url_launcher/url_launcher.dart';

Syntax:

```

class UrlLauncherExample extends StatelessWidget {
  _launchURL() async {
    const url = 'https://flutter.dev';
    if (await canLaunch(url)) {

```

```

        await launch(url);
    } else {
        throw 'Could not launch $url';
    }
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(title: Text('URL Launcher Example')),
        body: Center(
            child: ElevatedButton(
                onPressed: _launchURL,
                child: Text('Open Flutter Website'),
            ),
        ),
    );
}

```

REST API in Flutter

REST (Representational State Transfer) APIs are widely used in web and mobile applications to enable communication between a client (like a mobile app) and a server.

What is a REST API?

- **Stateless:** Each API call is independent and does not rely on previous calls.
- **Resource-Based:** Resources are identified by URLs, and operations are performed using standard HTTP methods (GET, POST, PUT, DELETE).
- **Format:** Responses are usually in JSON format, but can also be XML or others.

In Flutter, two popular packages for handling REST APIs are:

- **http**
- **dio**

1. http

A simple and easy-to-use package for making HTTP requests. It provides basic functionality to perform GET, POST, PUT, DELETE, and more.

HTTP Methods

- **GET**: Retrieve data from the server.
- **POST**: Send data to the server to create a new resource.
- **PUT**: Update an existing resource.
- **DELETE**: Remove a resource from the server.

You will gain a better understanding by implementing a project; here is an example of how to handle REST APIs using the HTTP package.

First, Create a flutter project and Add the http package to your pubspec.yaml file to handle HTTP requests

Step 1:

Create a Model.

```
// lib/models/user.dart
class User {
  final int id;
  final String name;
  final String email;

  User({required this.id, required this.name, required this.email});

  factory User.fromJson(Map<String, dynamic> json) {
    return User(
      id: json['id'],
      name: json['name'],
      email: json['email'],
    );
  }
}
```

Step 2:

Create a service class that will handle fetching data from the REST API.

```
// lib/services/user_service.dart
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../models/user.dart';

class UserService {
  final String baseUrl = 'https://jsonplaceholder.typicode.com/users';

  Future<List<User>> fetchUsers() async {
    final response = await http.get(Uri.parse(baseUrl));
```

```

    if (response.statusCode == 200) {
      List<dynamic> jsonData = json.decode(response.body);
      return jsonData.map((user) => User.fromJson(user)).toList();
    } else {
      throw Exception('Failed to load users');
    }
  }
}

```

Step 3:

Create a simple UI that fetches and displays the list of users.

```

// lib/views/user_list_view.dart
import 'package:flutter/material.dart';
import '../services/user_service.dart';
import '../models/user.dart';

class UserListView extends StatefulWidget {
  @override
  _UserListViewState createState() => _UserListViewState();
}

class _UserListViewState extends State<UserListView> {
  final UserService userService = UserService();
  late Future<List<User>> futureUsers;

  @override
  void initState() {
    super.initState();
    futureUsers = userService.fetchUsers();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('User List')),
      body: FutureBuilder<List<User>>(
        future: futureUsers,
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return Center(child: CircularProgressIndicator());
          } else if (snapshot.hasError) {
            return Center(child: Text('Error: ${snapshot.error}'));
          } else {
            final users = snapshot.data!;
            return ListView.builder(
              itemCount: users.length,
              itemBuilder: (context, index) {
                final user = users[index];
                return ListTile(

```

```

        title: Text(user.name),
        subtitle: Text(user.email),
      );
    },
  );
}
},
),
);
}
}

```

Step 4:

Update your main.dart to use the UserListView

// lib/main.dart

```
import 'package:flutter/material.dart';
```

```
import 'views/user_list_view.dart';
```

```

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'REST API Demo',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: UserListView(),
    );
  }
}

```

Here's how to fetch data from a REST API and display it in our UI section.

You can also expand this foundation by adding features like creating, updating, and deleting resources, as well as improving error handling and user experience.

We also have to learn about the Dio package for handling REST APIs,

2. dio

A powerful HTTP client for Dart that supports advanced features like interceptors, global configuration, form data, file downloading, and cancellation of requests.

Create a Flutter project and then add the Dio package to your pubspec.yaml file:

Step 1:

Create a Model

```
// lib/models/user.dart
class User {
  final int id;
  final String name;
  final String email;

  User({required this.id, required this.name, required this.email});

  factory User.fromJson(Map<String, dynamic> json) {
    return User(
      id: json['id'],
      name: json['name'],
      email: json['email'],
    );
  }
}
```

Step 2:

Create a Service Class Using Dio

```
// lib/services/user_service.dart
import 'dart:convert';
import 'package:dio/dio.dart';
import '../models/user.dart';

class UserService {
  final Dio _dio = Dio();
  final String baseUrl = 'https://jsonplaceholder.typicode.com/users';

  Future<List<User>> fetchUsers() async {
    try {
      final response = await _dio.get(baseUrl);
      List<dynamic> jsonData = response.data;
      return jsonData.map((user) => User.fromJson(user)).toList();
    } catch (e) {
      throw Exception('Failed to load users: $e');
    }
  }
}
```

Step 3:

Create a Simple UI

```
// lib/views/user_list_view.dart
import 'package:flutter/material.dart';
import '../services/user_service.dart';
```

```

import '../models/user.dart';

class UserListView extends StatefulWidget {
  @override
  _UserListViewState createState() => _UserListViewState();
}

class _UserListViewState extends State<UserListView> {
  final UserService userService = UserService();
  late Future<List<User>> futureUsers;

  @override
  void initState() {
    super.initState();
    futureUsers = userService.fetchUsers();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('User List')),
      body: FutureBuilder<List<User>>(
        future: futureUsers,
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return Center(child: CircularProgressIndicator());
          } else if (snapshot.hasError) {
            return Center(child: Text('Error: ${snapshot.error}'));
          } else {
            final users = snapshot.data!;
            return ListView.builder(
              itemCount: users.length,
              itemBuilder: (context, index) {
                final user = users[index];
                return ListTile(
                  title: Text(user.name),
                  subtitle: Text(user.email),
                );
              },
            );
          }
        },
      ),
    );
  }
}

```

Step 4:


```

Update Your main.dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'views/user_list_view.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'REST API Demo',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: UserListView(),
    );
  }
}

```

MediaQuery

MediaQuery provides information about the size and orientation of the current app's screen, allowing you to create responsive layouts.

Syntax:

```

class MediaQueryExample extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    final mediaQuery = MediaQuery.of(context);
    final isPortrait = mediaQuery.orientation == Orientation.portrait;

    return Scaffold(
      appBar: AppBar(title: Text('MediaQuery Example')),
      body: Center(
        child: Text(
          'Screen Width: ${mediaQuery.size.width}, Orientation: ${isPortrait ? "Portrait" : "Landscape"}',
          style: TextStyle(fontSize: 20),
        ),
      ),
    );
  }
}

```

GPS (Geolocation)

To use GPS functionality in Flutter, the geolocator package is often used to access device location.

Implementation Steps

- **Add Dependency:** Include the geolocator package in your pubspec.yaml:
geolocator: ^9.0.0
-
- **Import Package:** import 'package:geolocator/geolocator.dart';

Syntax:

```
class GpsExample extends StatefulWidget {  
  @override  
  _GpsExampleState createState() => _GpsExampleState();  
}
```

```
class _GpsExampleState extends State<GpsExample> {  
  String _locationMessage = "";
```

```
  @override  
  void initState() {  
    super.initState();  
    _getCurrentLocation();  
  }
```

```
  Future<void> _getCurrentLocation() async {  
    LocationPermission permission = await Geolocator.requestPermission();  
  
    if (permission == LocationPermission.denied) {  
      // Handle permission denied  
      return;  
    }  
  }
```

```
  Position position = await Geolocator.getCurrentPosition(desiredAccuracy: LocationAccuracy.high);  
  setState(() {  
    _locationMessage = "Location: ${position.latitude}, ${position.longitude}";  
  });  
}
```

```
@override  
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(title: Text('GPS Example')),  
    body: Center(  
      child: Text(_locationMessage),  
    ),  
  ),  
}
```

```
);  
}  
}
```

Nowadays, there's significant competition in application animations, making it essential to implement animations in every Flutter app. Flutter offers various animated widgets, and it's important to understand how to use them and why Flutter is advantageous for animations

Animated Widgets in Flutter

Animations enhance the user experience by providing visual feedback, guiding users through an app, and making transitions smoother. Flutter offers a rich set of animated widgets that simplify the process of adding animations to your applications.

Why Use Animated Widgets?

- **Enhanced User Experience:** Animations can make the app feel more responsive and interactive.
- **Visual Feedback:** Helps indicate state changes, such as button presses or loading states.
- **Smooth Transitions:** Improves the flow of the app, making transitions between screens or states more engaging.

Importance of Flutter for Animations

- **Built-in Animation Library:** Flutter provides a comprehensive set of widgets and classes for animations, including `AnimatedWidget`, `AnimatedBuilder`, and various transition widgets.
- **Declarative UI:** Flutter's declarative approach makes it easy to express complex animations through widget trees.
- **Performance:** Flutter's rendering engine is optimized for smooth animations and transitions, minimizing jank and ensuring high frame rates.
- **Hot Reload:** Facilitates quick iterations during development, allowing developers to see the results of animation changes in real-time.

Animated widgets can be categorized into several types, Such as;

- **AnimatedContainer**
- **FadeTransition**
- **ScaleTransition**

- **SlideTransition**
- **AnimatedOpacity**

Here's how we can utilize various types of animated widgets in different formats.

1. **AnimatedContainer**

The AnimatedContainer widget automatically transitions its properties over a given duration whenever they change.

Syntax:

```
AnimatedContainer(
  duration: Duration(seconds: 1),
  curve: Curves.easeInOut,
  width: _width,
  height: _height,
  color: _color,
  child: child,
)
```

2. **FadeTransition**

This widget fades its child in and out based on an Animation<double>.

Syntax:

```
FadeTransition(
  opacity: _animation,
  child: child,
)
```

3. **ScaleTransition**

Scales its child based on an Animation<double>.

Syntax:

```
ScaleTransition(
  scale: _animation,
  child: child,
)
```

4. **SlideTransition**

Moves its child based on an Animation<Offset>.

Syntax:

```
SlideTransition(
  position: _animation,
  child: child,
)
```

5. AnimatedOpacity

Allows you to animate the opacity of a widget over a specified duration.

Syntax:

```
AnimatedOpacity(
  opacity: _opacity,
  duration: Duration(seconds: 1),
  child: child,
)
```

You can also create custom animated widgets using `AnimatedBuilder` or by extending `ImplicitlyAnimatedWidget`.

Here is an Example of the Custom animated Widget

```
class MyAnimatedWidget extends ImplicitlyAnimatedWidget {
  final double width;

  MyAnimatedWidget({Key? key, required this.width}) : super(key: key, duration: Duration(seconds: 1));

  @override
  // Override the `for` method to build the child with the animation.
  Widget build(BuildContext context, Listenable listenable) {
    final animation = listenable as Animation<double>;
    return Container(
      width: width * animation.value,
      height: 100,
      color: Colors.blue,
    );
  }

  @override
  void forEachTween(TweenVisitor visitor) {
    // Define your tweens here
  }
}
```

Best Ways for Using Animated Widgets

- **Keep It Simple:** Overusing animations can lead to a cluttered UI. Use them where they add value.
- **Performance:** Avoid complex animations that can lead to performance issues. Use tools like the Flutter DevTools to monitor performance.
- **Consistency:** Maintain a consistent animation style throughout your app for a cohesive user experience.
- **User Control:** Allow users to control certain animations (e.g., play/pause) where appropriate.

Finally, we can create a project that implements Google Maps, which will help you understand how to work with the Google Maps API.

Setting up Google Maps in a Flutter app involves several steps, including configuring your project, adding dependencies, and writing the necessary code to display a map.

Step 1: Create a New Flutter Project

Step 2: Get Google Maps API Key

- Go to the Google Cloud Console.
- Create a new project (or select an existing one).
- Navigate to the "APIs & Services" dashboard.
- Enable the Maps SDK for Android and Maps SDK for iOS.
- Go to the "Credentials" tab and click on "Create credentials" > "API key".
- Restrict your API key for security purposes by setting the necessary restrictions for both Android and iOS.

Step 3: Configure Android

Open `android/app/src/main/AndroidManifest.xml`.

Add the following permission and metadata inside the `<application>` tag:

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>

<application>
  ...
  <meta-data
    android:name="com.google.android.geo.API_KEY"
    android:value="YOUR_API_KEY_HERE"/>
  ...
</application>
```

```
</application>
```

Replace YOUR_API_KEY_HERE with your actual API key.

Step 5: Add Dependencies

Add the google_maps_flutter dependency to your pubspec.yaml file:

dependencies:

flutter:

sdk: flutter

google_maps_flutter: ^2.1.1 # Check for the latest version

Step 6: Implement Google Maps in Your App

Open lib/main.dart and Replace the existing code with the following:

```
import 'package:flutter/material.dart';
```

```
import 'package:google_maps_flutter/google_maps_flutter.dart';
```

```
void main() {  
  runApp(MyApp());  
}
```

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Google Maps Demo',  
      theme: ThemeData(primarySwatch: Colors.blue),  
      home: MapScreen(),  
    );  
  }  
}
```

```
class MapScreen extends StatefulWidget {  
  @override  
  _MapScreenState createState() => _MapScreenState();  
}
```

```
class _MapScreenState extends State<MapScreen> {  
  late GoogleMapController mapController;  
  
  static final CameraPosition _initialPosition = CameraPosition(  
    target: LatLng(37.7749, -122.4194), // San Francisco  
    zoom: 10,  
  );  
  
  void _onMapCreated(GoogleMapController controller) {  
    mapController = controller;  
  }  
}
```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text('Google Maps in Flutter')),
    body: GoogleMap(
      onMapCreated: _onMapCreated,
      initialCameraPosition: _initialPosition,
    ),
  );
}

```

You can also use the Marker class provided by the google_maps_flutter package;

Syntax:

```

import 'package:flutter/material.dart';
import 'package:google_maps_flutter/google_maps_flutter.dart';

```

```

void main() {
  runApp(MyApp());
}

```

```

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Google Maps Demo',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: MapScreen(),
    );
  }
}

```

```

class MapScreen extends StatefulWidget {
  @override
  _MapScreenState createState() => _MapScreenState();
}

```

```

class _MapScreenState extends State<MapScreen> {
  late GoogleMapController mapController;

  static final CameraPosition _initialPosition = CameraPosition(
    target: LatLng(37.7749, -122.4194), // San Francisco
    zoom: 10,
  );

```

```

  final Set<Marker> _markers = {}; // Set to hold markers

```



```

void _onMapCreated(GoogleMapController controller) {
  mapController = controller;

  // Add an initial marker
  _addMarker(LatLng(37.7749, -122.4194), 'San Francisco');
}

void _addMarker(LatLng position, String title) {
  final String markerId = position.toString();
  _markers.add(
    Marker(
      markerId: MarkerId(markerId),
      position: position,
      infoWindow: InfoWindow(
        title: title,
        snippet: 'A popular city in California.',
      ),
    ),
  );
  setState(() {}); // Refresh the UI
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text('Google Maps in Flutter')),
    body: GoogleMap(
      onMapCreated: _onMapCreated,
      initialCameraPosition: _initialPosition,
      markers: _markers, // Pass the markers to the GoogleMap
    ),
  );
}

```

What we used on here is

- **Markers Set:** A Set<Marker> named _markers is created to hold the markers on the map.
- **Adding an Initial Marker:** In the _onMapCreated method, we call _addMarker to place a marker at the initial position (San Francisco).
- **_addMarker Method:** This method creates a new Marker and adds it to the _markers set. It includes a unique markerId, position, and an infoWindow with a title and snippet.
- **Markers Property:** The markers property of the GoogleMap widget is set to the _markers set, which updates the map to display the markers.

Make sure your device or emulator is running, after all of this run your app.